

Eval Function Explanation

Here's the selected text in a writing block:



variable

```
class Number:
    def __init__(self, n):
        self.n = n

    # def __add__(self, num):
    #     return self.n + num.n # Access the 'n' attribute of
    # the 'num' object

n = Number(1)
m = Number(2)
```

When you write:

```
<> Python
```

```
print n + m
```

Python internally calls:

```
<> Python
```

```
n.      m
```

So if you comment out `__add__()`, Python does not know how to add two `Number` objects and will raise:

```
<> Python
```

```
TypeError unsupported operand s for + 'Number' and 'Number'
```

Why?

Python knows how to add built-in types:

</> Python

```
1 + 2
"a" + "b"
1 + 2
```

because those classes already have an `__add__()` method.

Your custom class does not.

Solution 1: Access the attributes directly

</> Python

```
class Number
    def __init__ self n
        self. = n

n = Number 1
m = Number 2

print n. + m.
```

Output:

</> Python

```
3
```

Here you're adding the integer values stored inside the objects.

Solution 2: Create a normal method

</> Python

```
class Number
    def __init__ self n
        self. = n

    def add self other
        return self. + other.

n = Number 1
m = Number 2

print n. m
```

Output:

</> Python

3

Solution 3: Using a getter method

</> Python

```
class Number
    def __init__ self n
        self. = n

    def getValue self
        return self.

n = Number 1
m = Number 2

print n. + m.
```

Output:

</> Python

3

Can we still use `n + m` without `__add__()` ?

No.

The `+` operator for custom objects requires operator overloading through:

</> Python

```
def __add__ self other
```

Without `__add__()` , Python has no instruction for what `n + m` means.

So:

</> Python

```
print n + m
```

✗ Error

but

</> Python

```
print n. + m.  Works
```

or

</> Python

```
print n. m  Works
```

Interview Answer

The `+` operator on custom objects works through operator overloading using the special method `__add__()`. If `__add__()` is not defined, Python cannot perform `n + m` and raises a `TypeError`. To add the values without `__add__()`, we must explicitly access the object's attributes (`n.n + m.n`) or use a regular method such as `add()`.

explain how it works class Number:

```
def __init__(self, n):
    self.n = n
```

```
def __add__(self, num):
    return self.n + num.n # Access the 'n' attribute of the
                           'num' object
```

```
n = Number(1)
m = Number(2)
```

This is an example of **Operator Overloading** in Python.

Let's understand it line by line.

</> Python

```
class Number
    def __init__ self n
        self. = n

    def __add__ self num
        return self. + num.

n = Number 1
```

```
m = Number 2

print n + m
```

Step 1: Class Creation

</> Python

```
class Number
```

A new class named `Number` is created.

Think of it as a blueprint for creating objects.

Step 2: Constructor Execution

</> Python

```
def __init__ self n
    self. = n
```

`__init__()` is automatically called whenever an object is created.

Creating First Object

</> Python

```
n = Number 1
```

Python internally does:

</> Python

```
n = Number.      Number
n.      1
```

Inside `__init__()`:

</> Python

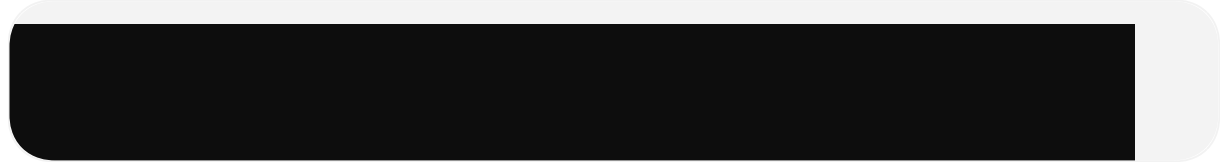
```
self = n
n = 1
```

Therefore:

```
</> Python
```

```
self. = 1
```

Object becomes:



Creating Second Object

```
</> Python
```

```
m = Number 2
```

Inside constructor:

```
</> Python
```

```
self = m
```

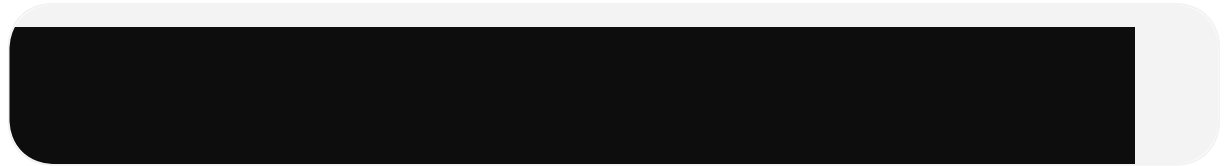
```
n = 2
```

So:

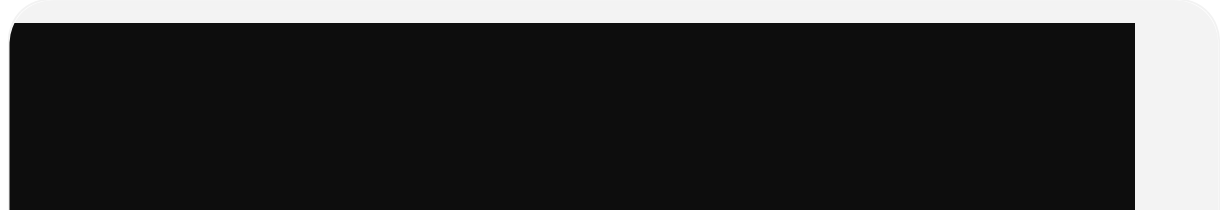
```
</> Python
```

```
self. = 2
```

Object becomes:



Memory Representation



Step 3: What Happens During `n + m`?

```
</> Python
```

```
print n + m
```

Python does **not** directly add the objects.

Instead, it translates:

```
</> Python
```

```
n + m
```

into

```
</> Python
```

```
n.      m
```

This is called **Operator Overloading**.

Step 4: Method Call

Python executes:

```
</> Python
```

```
n.      m
```

Now:

```
</> Python
```

```
self = n  
num = m
```

Inside the method:

```
</> Python
```

```
def __add__ self num  
    return self. + num.
```

becomes:

```
</> Python
```

```
return n. + m.
```

Step 5: Accessing Values

From memory:

```
</> Python
```

```
n. = 1  
m. = 2
```

Therefore:

```
</> Python
```

```
return 1 + 2
```

Result:

```
</> Python
```

```
3
```

Step 6: Print Output

```
</> Python
```

```
print 3
```

Output:

Complete Dry Run

</> Python

```
n = Number 1
```

Object:



</> Python

```
m = Number 2
```

Object:



</> Python

```
n + m
```

Python converts to:

</> Python

```
n.      m
```

Now:

</> Python

```
self = n  
num = m
```

Inside method:

</> Python

```
return self. + num.
```

↓

</> Python

```
return 1 + 2
```

↓

<> Python

```
return 3
```

↓

<> Python

```
print 3
```

Output:

Why `num.n`?

Inside:

<> Python

```
def __add__ self num
```

`num` is not a number.It is the **second object**.

For:

<> Python

```
n + m
```

we get:

<> Python

```
self = n  
num = m
```

So:

<> Python

```
num.
```

means:

```
</> Python
```

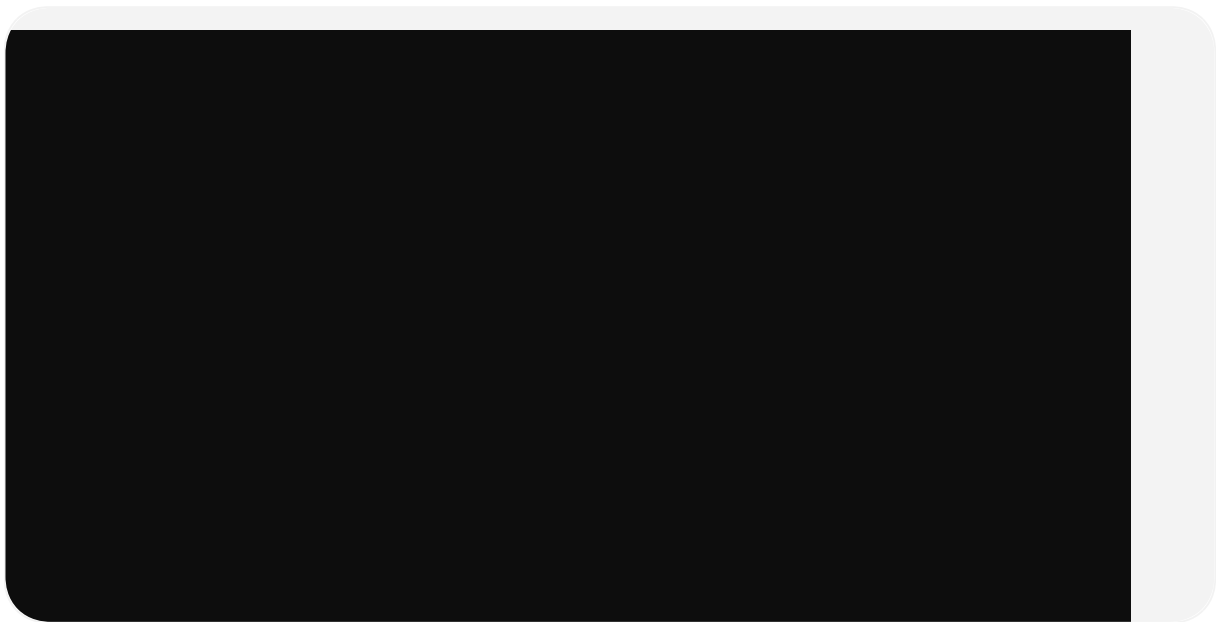
```
m.
```

which is:

```
</> Python
```

```
2
```

Visual Representation



Important Interview Point

Python operators are actually method calls:

Operator	Special Method
+	<code>__add__()</code>
-	<code>__sub__()</code>
*	<code>__mul__()</code>
/	<code>__truediv__()</code>

Operator	Special Method
<code>==</code>	<code>__eq__()</code>
<code><</code>	<code>__lt__()</code>
<code>></code>	<code>__gt__()</code>

So:

`</>` Python

```
n + m
```

is internally:

`</>` Python

```
n. m
```

and that's why defining `__add__()` allows custom objects to be added using the `+` operator.

what is enumerate funtion in python? l = [3, 513, 53, 535]

```
# index = 0
# for item in l:
#     print(f"The item number at index {index} is {item}")
#     index += 1
```

#NEW WAY

This can be simplified using enumerate function

#List comprehension.

```
for index, item in enumerate(l):
```

`enumerate()` is a built-in Python function that allows you to get **both the index and the value** of an iterable (like a list, tuple, string, etc.) while looping through it.

Without `enumerate()`

You were manually maintaining the index:

</> Python

```
l = 3 513 53 535

index = 0
for item in l:
    print f"The item number at index {index} is {item}"
    index += 1
```

Dry Run

Iteration 1

</> Python

```
index = 0
item = 3
```

Output:

```
The item number at index 0 is 3
```

Then:

</> Python

```
index += 1
```

becomes:

</> Python

```
index = 1
```

Iteration 2

</> Python

```
item = 513
```

Output:

```
The item number at index 1 is 513
```

And so on...

With `enumerate()`

</> Python

```
l = 3 513 53 535

for index item in enumerate l
    print f"The item number at index index is item "
```

What does `enumerate(l)` return?

</> Python

```
enumerate l
```

creates an enumerate object that generates:

</> Python

```
0 3
1 513
2 53
3 535
```

So Python sees:

</> Python

```
for index item in enumerate l
```

as:

</> Python

```
for index item in
    0 3
    1 513
    2 53
    3 535
```

Dry Run

Iteration 1

</> Python

```
index item = 0 3
```

Therefore:

</> Python

```
index = 0  
item = 3
```

Output:

Iteration 2

</> Python

```
index item = 1 513
```

Therefore:

</> Python

```
index = 1  
item = 513
```

Output:

Iteration 3

</> Python

```
index item = 2 53
```

Output:

Iteration 4

```
<> Python
```

```
index item = 3 535
```

Output:

Final Output

Starting from a Different Index

By default, indexing starts at `0`.

You can change it:

```
<> Python
```

```
l = 3 513 53 535
```

```
for index item in enumerate(l, start=1):  
    print index, item
```

Output:

Other Examples

String

```
<> Python
```



```
name = "Harry"

for index ch in enumerate name
    print index ch
```

Output:

Tuple

<> Python

```
t = 10 20 30

for index value in enumerate t
    print index value
```

Output:

What Does `enumerate()` Actually Return?

<> Python

```
l = 3 513 53 535

e = enumerate l

print list e
```

Output:

<> Python

```
0 3 1 513 2 53 3 535
```

Each element is a tuple:

```
<> Python
```

```
index value
```

Why Use `enumerate()` ?

Instead of:

```
<> Python
```

```
index = 0
for item in l:
    print index, item
    index += 1
```

use:

```
<> Python
```

```
for index, item in enumerate(l):
    print index, item
```

Benefits:

- Cleaner code
- No need to manually update the index
- Less chance of bugs
- More Pythonic

Interview Answer

`enumerate()` is a built-in Python function that adds a counter to an iterable and returns an enumerate object. During iteration, it provides both the index and the value of each element. It is commonly used when we need the position and the element at the same time, avoiding manual index management. For example, `enumerate([10, 20, 30])` produces `(0,10)`, `(1,20)`, and `(2,30)`.